

Resumen teórico

Testing de Software — TP2

Índice

Introducción	3
1. Por qué automatizar el testing	3
1.1. Testing <i>ad hoc</i> y sus límites	3
1.2. Definición de <i>Test automation</i>	3
2. Testeabilidad, observabilidad y controlabilidad	4
2.1. Testeabilidad	4
2.2. Observabilidad	4
2.3. Controlabilidad	4
2.4. Intuición práctica	5
3. Componentes de un test case	5
3.1. Test case values	5
3.2. Prefix values	5
3.3. Postfix values	5
3.4. Expected results y test oracle	5
3.5. Test case y test set	6
3.6. Relación con el modelo RIPR	6
4. JUnit como framework de automatización	6
4.1. Qué provee JUnit	6
4.2. Estructura básica de un test JUnit	7
4.3. Aserciones más usadas	7
4.4. Aserciones agrupadas	7
4.5. Matchers Hamcrest y <code>assertThat</code>	8
4.6. Características deseables de un test	8

5. Suites, fixtures y ciclo de vida	8
5.1. Test suites	8
5.2. Fixtures	9
5.3. setUp y tearDown	9
5.4. Fixtures globales	9
5.5. Timeouts	10
6. Tests negativos	10
6.1. Definición	10
6.2. Por qué importan	10
6.3. Ejemplo extendido: Min.min	11
7. Data-driven testing y tests parametrizados	11
7.1. Motivación	11
7.2. Idea central	12
7.3. @ValueSource	12
7.4. @CsvSource	12
7.5. @CsvFileSource	12
7.6. @MethodSource	13
7.7. Qué método elegir	13
7.8. Conversión de tipos en CSV	14
7.9. Ventajas del enfoque data-driven	14
7.10. Riesgo a vigilar	14
8. Cómo se conectan estas ideas con el TP2	14
9. Síntesis	15

Introducción

Este documento es un resumen teórico de los contenidos asociados al **Práctico 2** de *Testing de Software* de la UNRC. Las fuentes principales son tres y se complementan entre sí:

- el **capítulo 3** de *Introduction to Software Testing* (P. Ammann y J. Offutt, 2ª edición), titulado *Test Automation*;
- las **notas 03 “Automatización del testing”** (Valeria Bengolea), que recorren el mismo capítulo enfocándose en JUnit, *fixtures* y tests negativos;
- las **notas 04 “Data-Driven Tests”**, que introducen los tests parametrizados de JUnit 5.

La idea del resumen es presentar los conceptos en un hilo claro: por qué automatizamos, qué hace falta para que un test sea automatizable (*testability*, *observability*, *controllability*), cuál es la estructura formal de un test, cómo se traduce a JUnit, cómo se organiza una suite, qué son los tests negativos y, finalmente, cómo se generalizan a tests *data-driven* para evitar duplicación. Cada sección incluye ejemplos breves para que la lectura sea fácil de retener.

1. Por qué automatizar el testing

1.1. Testing *ad hoc* y sus límites

La forma más natural de probar software es *ad hoc*: el programador escribe un `main` que invoca su función y observa la salida en consola. Funciona en aplicaciones con una interfaz directa y muestras chicas, pero deja de escalar muy rápido. Tres limitaciones aparecen casi de inmediato:

- **no almacena los tests** para ejecuciones futuras: cada vez que se cambia el código hay que volver a inventarlos;
- para probar funcionalidades **internas**, que no son invocables desde la línea de comandos, hace falta programar un *arnés* (un `main` ad hoc que recibe argumentos y llama al método);
- requiere **inspección humana** para decidir si la salida es correcta: alguien tiene que mirar el resultado y compararlo mentalmente con el valor esperado.

1.2. Definición de *Test automation*

Ammann y Offutt definen la automatización del testing así (Definición 3.9):

Test automation: el uso de software para controlar la ejecución de tests, comparar resultados obtenidos con los esperados, setear las precondiciones del test y otras tareas de control y reporte.

Automatizar trae cuatro beneficios concretos:

- **reduce costos**: una vez escrito, un test se ejecuta sin esfuerzo humano adicional;
- **reduce errores de inspección**: la comparación queda en manos del framework, no de los ojos del programador;
- **habilita *regression testing***: el conjunto de tests acumulado se vuelve un seguro contra regresiones cada vez que se modifica el código;
- **permite correr la misma suite muchas veces** con poco costo marginal: por ejemplo, en cada *push* a un repositorio.

2. Testeabilidad, observabilidad y controlabilidad

2.1. Testeabilidad

Definición 3.10:

Testability: el grado en que un sistema o componente facilita el establecimiento de criterios de testing y la ejecución de los tests para determinar si dichos criterios se cumplen.

La testeabilidad de un programa depende, fundamentalmente, de dos preguntas:

- **¿cómo se proporcionan valores a la pieza de software bajo prueba?**
- **¿cómo se observan los resultados de su ejecución?**

2.2. Observabilidad

Definición 3.11: facilidad con la que se observa el comportamiento de un programa en términos de sus salidas, efectos en el entorno y otras componentes. Software que afecta dispositivos hardware, bases de datos remotas o ficheros suele tener **baja observabilidad**: el efecto está, pero no es directamente visible para el test.

2.3. Controlabilidad

Definición 3.12: facilidad con la que se proveen al programa los *inputs* necesarios, en términos de valores, operaciones y comportamientos. El software que recibe datos por

teclado o por un parámetro de método se controla fácilmente; el software que recibe inputs de sensores físicos, en cambio, suele ser muy difícil de poner en un estado específico.

2.4. Intuición práctica

Cuando la observabilidad o la controlabilidad son bajas, una técnica habitual es la **simulación**: introducir software extra (mocks, drivers, stubs) que reemplaza al componente difícil y le da al test una API más amigable. Es una forma de mejorar la testeabilidad sin modificar el sistema bajo prueba.

3. Componentes de un test case

Un test no es sólo *una entrada*: tiene varias piezas. El capítulo 3 las formaliza con definiciones que conviene tener a mano (Definiciones 3.13 a 3.20).

3.1. Test case values

Los **valores del test** (*test case values*) son los *inputs* que satisfacen los *test requirements*. Son los datos directos que ejercitan la rutina bajo prueba.

3.2. Prefix values

Los **valores prefijos** son los inputs necesarios para llevar al software al estado apropiado para recibir los valores del test. En términos del modelo RIPR, son los responsables del paso *Reachability*: gracias a ellos, el flujo de ejecución alcanza el código defectuoso. Ejemplo: para testear `Stack.pop()` hay que primero hacer **push** de algunos elementos.

3.3. Postfix values

Los **valores postfixos** son los inputs que hay que enviar al software *después* de los valores del test. Se subdividen en:

- **Verification values**: necesarios para *ver* el resultado (p. ej. consultar el tope de la pila después de un **push**);
- **Exit values**: comandos para terminar el programa o devolverlo a un estado estable (p. ej. cerrar una conexión a base de datos).

3.4. Expected results y test oracle

El **resultado esperado** es el valor que debería producir la rutina si se comporta correctamente. La pieza que decide si el test pasó o falló se llama **oráculo** y, en frameworks

como JUnit, se materializa en aserciones (`assertEquals`, `assertThat`, etc.).

3.5. Test case y test set

- Definición 3.19. Un **test case** es la combinación de valores del test, valores prefijos, valores postfijos y resultados esperados, suficiente para una ejecución y evaluación completa del software bajo prueba.
- Definición 3.20. Un **test set** (o *test suite*) es un conjunto de test cases.

3.6. Relación con el modelo RIPR

Los componentes del test case son realizaciones concretas del modelo **RIPR** visto en el TP1:

- los *prefix values* apuntan a la **Reachability**,
- los *test case values* apuntan a la **Infection**,
- los *postfix values* apuntan a la **Propagation**,
- los *expected results* apuntan a la **Revealability**.

4. JUnit como framework de automatización

4.1. Qué provee JUnit

JUnit es el framework estándar de testing automatizado para Java. Sus capacidades principales son:

- define una **estructura estándar** de test, basada en anotaciones (`@Test`, `@BeforeEach`, etc.);
- permite almacenar los tests como **scripts ejecutables**;
- organiza tests en **suites**;
- provee **aserciones** para chequear automáticamente los resultados esperados;
- ofrece **entornos de ejecución** (Maven Surefire, IDE) y **reportes detallados** de las pruebas que fallan.

Está orientado a tests **unitarios** y **de integración**; no es la herramienta ideal para tests de sistema o de UI. Lenguajes modernos tienen frameworks análogos: pytest para Python, RSpec para Ruby, JUnit para Java/Kotlin.

4.2. Estructura básica de un test JUnit

```
1 @Test
2 public void testMax() {
3     // arrange: preparar los datos de entrada
4     int a = 1;
5     int b = 3;
6     // act: ejecutar la rutina bajo prueba
7     int res = SimpleRoutines.max(a, b);
8     // assert: comparar contra el resultado esperado
9     assertTrue(res == 3);
10 }
```

El patrón *arrange-act-assert* no es una imposición del framework, pero es la convención dominante porque separa de un vistazo las tres responsabilidades del test.

4.3. Aserciones más usadas

JUnit incluye un conjunto rico de aserciones; las más habituales:

- `assertTrue(expr) / assertFalse(expr);`
- `assertEquals(esperado, actual);`
- `assertArrayEquals(a, b);`
- `assertNotNull(o) / assertNull(o);`
- `assertSame(o1, o2) / assertNotSame(o1, o2);`
- `assertThrows(Exc.class, () ->...) para tests negativos;`
- `assertTimeoutPreemptively(d, () ->...) para evitar cuelgues.`

4.4. Aserciones agrupadas

JUnit 5 permite agrupar varias aserciones con `assertAll`, de modo que se reporten *todas* las fallas y no sólo la primera:

```
1 @Test
2 void groupedAssertions() {
3     assertAll("person",
4         () -> assertEquals("Jane", person.getFirstName()),
5         () -> assertEquals("Doe", person.getLastName())
6     );
7 }
```

4.5. Matchers Hamcrest y `assertThat`

Las versiones recientes proveen una API declarativa basada en *matchers*:

```
1 assertThat(1, hasItems(2, 3));
2 assertThat(list, everyItem(greaterThan(1)));
3 assertThat("HoLa", equalToIgnoringCase("hola"));
4 assertThat(obj, instanceof(Student.class));
```

Los matchers se agrupan en familias (*logical*, *collections*, *text*, *numbers*, etc.) y permiten escribir aserciones más expresivas que un simple `equals`. Una alternativa popular en el mismo espacio es `AssertJ`.

4.6. Características deseables de un test

Independientemente del framework, un buen test debe ser:

- **automático**: ejecutable con un solo comando, sin intervención manual;
- **repetible**: produce el mismo resultado en distintas ejecuciones;
- **fácil de implementar y de leer**;
- **independiente** de otros tests: no debe asumir orden de ejecución;
- **rápido** y eficiente en uso de recursos.

5. Suites, fixtures y ciclo de vida

5.1. Test suites

Una *test suite* es simplemente un conjunto de test cases. En JUnit 5 se declara con `@Suite` y `@SelectClasses`:

```
1 @Suite
2 @SelectClasses({
3     UnitTests1.class,
4     UnitTests2.class,
5     ModuleTests.class,
6     IntegrationTests.class
7 })
8 class AllUnitTest { }
```


5.2. Fixtures

Un **fixture** define el estado inicial de un test. Es útil cuando varios tests comparten datos o necesitan inicializaciones costosas. Se implementa declarando los objetos como atributos de la clase de test e inicializándolos en un método anotado con `@BeforeEach`.

```
1 public class MinTest {
2     private List<String> list;           // test fixture
3
4     @BeforeEach                         // setUp -- antes de
5         cada test
6     public void setUp() {
7         list = new ArrayList<String>();
8     }
9
10    @AfterEach                           // tearDown -- despues
11        de cada test
12    public void tearDown() {
13        list = null;
14    }
15 }
```

5.3. setUp y tearDown

Las rutinas `setUp` (`@BeforeEach`) y `tearDown` (`@AfterEach`) ayudan a garantizar la **independencia** entre tests:

- `@BeforeEach` se ejecuta antes de cada test de la suite;
- `@AfterEach` se ejecuta después de cada test, incluso si el test lanza una excepción.

Esto es importante porque JUnit no garantiza un orden de ejecución entre tests, y compartir estado mutable lleva a tests *flaky*.

5.4. Fixtures globales

Para operaciones costosas (abrir una conexión a base de datos, levantar un servidor mock) conviene ejecutar el setup una sola vez. JUnit 5 ofrece `@BeforeAll` y `@AfterAll`:

```
1 @BeforeAll
2 public static void openConnection() {
3     connection = DB.open(...);
4 }
5
```

```

6 @AfterAll
7 public static void closeConnection() {
8     connection.close();
9 }

```

5.5. Timeouts

Algunos tests ejecutan funcionalidades que podrían *no terminar* o demandar demasiado tiempo. Para protegerse, JUnit 5 ofrece la anotación `@Timeout` y la aserción `assertTimeoutPreemptively`:

```

1 @Test
2 @Timeout(value = 1000, unit = TimeUnit.MILLISECONDS)
3 public void test1() {
4     long res = Fibonacci.fibonacciEfi(45);
5     assertEquals(res, 1134903170);
6 }

```

Esto es especialmente útil para defectos como el *Zune bug*, donde una rama del control de flujo puede entrar en loop infinito.

6. Tests negativos

6.1. Definición

Los **tests negativos** son aquellos que ejercitan al software bajo condiciones que **violan su precondition** o que están fuera del comportamiento “feliz”. Suelen verificar que la rutina lanza la excepción correcta:

```

1 @Test
2 public void test1() {
3     BoundedQueue q = new BoundedQueue();
4     assertThrows(IllegalStateException.class, () -> q.dequeue());
5 }

```

6.2. Por qué importan

Son tan importantes como los tests positivos:

- documentan ejecutablemente las precondiciones del método;
- evitan regresiones cuando se modifica el código de validación;

- se conectan directamente con la noción de *contract* de la programación por contrato.

6.3. Ejemplo extendido: Min.min

El método `Min.min(List)` admite tres tipos de excepciones, y eso lleva naturalmente a siete tests:

1. lista `null` → `NullPointerException`
2. lista con `null` entre varios elementos → `NullPointerException`
3. lista con un único elemento `null` → `NullPointerException`
4. elementos no comparables (`Integer + String`) → `ClassCastException`
5. lista vacía → `IllegalArgumentException`
6. lista de un solo elemento → ese elemento
7. lista de dos elementos → el menor

Los primeros cinco son negativos; los dos últimos son tests del *happy path*. Esta combinación, planeada explícitamente, da una cobertura razonable del contrato del método.

7. Data-driven testing y tests parametrizados

7.1. Motivación

Una vez que aprendimos JUnit “básico” aparece un problema recurrente: muchos tests **tienen exactamente la misma estructura** y difieren solamente en los datos. Por ejemplo, para verificar `isLeapYear` habría que escribir:

```
1 @Test void isLeapYear_2000() { assertTrue(isLeapYear(2000)); }
2 @Test void isLeapYear_1980() { assertTrue(isLeapYear(1980)); }
3 @Test void isLeapYear_1992() { assertTrue(isLeapYear(1992)); }
4 @Test void notLeapYear_2001() { assertFalse(isLeapYear(2001)); }
5 // ...
```

La duplicación de código es evidente. La pregunta natural es: “¿podemos correr el mismo test sobre cada tupla de un conjunto de valores?”. La respuesta es **sí**: a esa técnica se la llama **data-driven testing**, y JUnit 5 la soporta con la familia `@ParameterizedTest`.

7.2. Idea central

Un test parametrizado separa la **estructura del test** (qué hace) de los **datos** (con qué valores se lo ejecuta). El framework se encarga de instanciar el test una vez por cada tupla del proveedor.

7.3. @ValueSource

La forma más simple. Recibe un arreglo de valores de un tipo primitivo o `String` y los pasa de a uno como argumento:

```
1 @ParameterizedTest
2 @ValueSource(ints = {2000, 1980, 1992})
3 void isLeapYear_ShouldReturnTrueForLeapYear(int year) {
4     assertTrue(SimpleRoutines.isLeapYear(year));
5 }
```

Limitaciones: sólo permite un argumento y sólo tipos primitivos o `String`.

7.4. @CsvSource

Permite varios argumentos en cada fila. Es ideal cuando el caso de prueba se puede expresar como “*x* produce *y*”:

```
1 @ParameterizedTest
2 @CsvSource({"2000,true", "1980,true", "1992,true",
3            "2001,false", "2023,false", "1994,false"})
4 void isLeapYearTest(int year, boolean expected) {
5     assertThat(SimpleRoutines.isLeapYear(year), is(expected));
6 }
```

JUnit hace conversión automática de tipos: “true” → `true`, “15” → `15`, “1.0” → `1.0d`, etc.

7.5. @CsvFileSource

Es igual que `@CsvSource` pero lee los datos de un archivo externo, ubicado dentro de `src/test/resources`. Resulta útil cuando hay muchos casos o cuando quiere editarlos alguien que no toca el código Java:

```
1 @ParameterizedTest
2 @CsvFileSource(resources = "leapYear.csv", numLinesToSkip = 1)
3 void isLeapYearTest_2(int year, boolean expected) {
4     assertThat(SimpleRoutines.isLeapYear(year), is(expected));
5 }
```

```
5 }
```

con un archivo `leapYear.csv`:

```
Year,Expected
2000,true
1980,true
1992,true
2001,false
```

7.6. @MethodSource

Cuando los argumentos no son tipos simples (por ejemplo, arreglos de enteros u objetos), un CSV resulta poco práctico. `@MethodSource` apunta a un método estático que devuelve un `Stream<Arguments>`:

```
1 private static Stream<Arguments> arraysProvider() {
2     return Stream.of(
3         Arguments.of(new int[]{7,8,9}, new int[]{7,8,9}),
4         Arguments.of(new int[]{9,8,7}, new int[]{7,8,9}),
5         Arguments.of(new int[]{1}, new int[]{1}),
6         Arguments.of(new int[]{}, new int[]{})
7     );
8 }
9
10 @ParameterizedTest(name = "{index}: sorting({0}) = {1}")
11 @MethodSource("arraysProvider")
12 public void testBubbleSort(int[] array, int[] sortedarray) {
13     Sorting.bubbleSort(array);
14     assertArrayEquals("Sorted", sortedarray, array);
15 }
```

Cada tupla de `Arguments.of(...)` se convierte en un test individual.

7.7. Qué método elegir

Situación	Proveedor recomendado
1 argumento, tipo primitivo o String	<code>@ValueSource</code>
varios argumentos primitivos / String, casos pocos	<code>@CsvSource</code>
varios argumentos primitivos / String, casos muchos	<code>@CsvFileSource</code>
arreglos, objetos, tipos compuestos	<code>@MethodSource</code>

7.8. Conversión de tipos en CSV

JUnit 5 convierte automáticamente el texto de cada celda al tipo declarado en el parámetro. Las conversiones más importantes son:

Tipo Java	Ejemplo
Boolean/boolean	"true" $\Rightarrow$ true
Character/char	".o" $\Rightarrow$ 'o'
Integer/int	"15" $\Rightarrow$ 15
Float/float	"1.0" $\Rightarrow$ 1.0f
Double/double	"1.0" $\Rightarrow$ 1.0d

Cuando JUnit no sabe cómo convertir un tipo (por ejemplo, `int[]`), hay que pasar a `@MethodSource` o usar un `ArgumentConverter` a medida.

7.9. Ventajas del enfoque data-driven

- **Menos código:** una sola declaración del cuerpo del test y todos los datos del lado de afuera.
- **Reporte por caso:** cada tupla se reporta como un test independiente, así que se ve exactamente qué entrada falló.
- **Facilita el cubrimiento:** agregar un caso de borde es agregar una fila, no escribir un test nuevo.
- **Separación de responsabilidades:** los datos de los casos pueden vivir en archivos editables por gente que no es desarrolladora.

7.10. Riesgo a vigilar

La tentación de meter “muchos casos” es real. Conviene seguir eligiendo los casos con criterio (clases de equivalencia, valores de borde, RIPR) y no por inercia: un test parametrizado con 200 filas redundantes es tan inútil como un test sin ninguna aserción real.

8. Cómo se conectan estas ideas con el TP2

El práctico 2 aterriza la teoría de este resumen en seis ejercicios:

- **Ejercicio 2** es el caso testigo del *data-driven testing*: los tests del TP1 sobre `SimpleRoutines` se reescriben con `@CsvSource`, `@MethodSource` y `@CsvFileSource`, mostrando los tres mecanismos en una sola suite.

- **Ejercicio 3** aplica todo el ciclo de vida JUnit (`@BeforeEach`, *arrange-act-assert*, aserciones de excepción) sobre `StackAr` y agrega el invariante de representación (`repOk`) como oráculo interno.
- **Ejercicio 4** muestra el patrón “casos válidos vs. casos inválidos” en dos CSV separados, e ilustra el rol de los tests negativos para las tres excepciones de `Min.min`.
- **Ejercicio 5** usa tests parametrizados junto con un *oráculo* alternativo, basado en `LocalDate.plusDays`, para hacer *differential testing* sobre `ZuneBug`, y emplea `assert-TimeoutPreemptively` para protegerse del loop infinito que producía la versión defectuosa.
- **Ejercicio 6** repite la estructura del Ejercicio 3 pero sobre una cola circular, mostrando que el invariante `back == (front + size) % capacity` hace de oráculo barato y muy útil después de cada operación.

9. Síntesis

Si tuviera que resumir el capítulo y las notas en una página, diría que en TP2 aprendimos a hacer cinco cosas que antes hacíamos a mano:

1. **Automatizar el oráculo** (con aserciones) en lugar de inspeccionar la salida con los ojos.
2. **Aislar el estado** entre tests usando `@BeforeEach` y, cuando hace falta, `@BeforeAll`.
3. **Cubrir el contrato** del método, no sólo el *happy path*, escribiendo tests negativos que prueban las precondiciones con `assertThrows`.
4. **Eliminar la duplicación** en suites repetitivas con `@ParameterizedTest` y sus distintos proveedores (`@ValueSource`, `@CsvSource`, `@CsvFileSource`, `@MethodSource`).
5. **Usar el invariante de representación** como oráculo interno: un `repOk` que se verifica después de cada operación amplifica significativamente la capacidad de detección de la suite.

Estos cinco gestos son la base sobre la que el resto del cuatrimestre construye: criterios de cobertura, mutation testing y model-based testing serán formas distintas de *generar* los casos, pero la maquinaria de ejecutarlos y evaluarlos sigue siendo la que vimos aquí.